



Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ ИНФОРМАТИКА И СИСТЕМЫ УПРАВЛЕНИЯ
КАФЕДРА СИСТЕМЫ ОБРАБОТКИ ИНФОРМАЦИИ И УПРАВЛЕНИЯ

ИНДИВИДУАЛЬНОЕ ДОМАШНЕЕ ЗАДАНИЕ № 2

«Разработка консольного приложения с применением ООП»

ДИСЦИПЛИНА: «Архитектура автоматизированных систем обработки информации и управления»

Выполнил: студент гр. ИУ5-23Б

_____ (Коротенко А.А.)
(Подпись) (Ф.И.О.)

Проверил:

_____ (Гапанюк Ю.Е.)
(Подпись) (Ф.И.О.)

Дата сдачи (защиты):

Результаты сдачи (защиты):

- Балльная оценка:
- Оценка:

Москва, 2026 г.

ОГЛАВЛЕНИЕ

Цель работы:	3
Задачи:	3
ВЫПОЛНЕНИЕ РАБОТЫ	4
1. Теоретический материал к заданию.....	Ошибка! Закладка не определена.
2. Реализация алгоритма	4
ПРИЛОЖЕНИЕ А	17

Цель работы:

Создание полноценного консольного приложения на C# на основе классов, способного работать с отдельными файлами и базами данных.

Задачи:

- Создание CSV-файлов с тестовыми данными
- Создание классов для каждой таблицы
- Создание класса DatabaseManager для взаимодействие с SQLite
- Реализовать консольное меню
- Реализовать три отчёта с помощью ReportBuilder
- Выполнить задание по варианту
- Нарисовать необходимые схемы

ВЫПОЛНЕНИЕ РАБОТЫ

1. Описание предметной области и создание CSV-файлов

Предметная область по варианту:

- Справочная таблица: Музеи (идентификатор и название)
- Основная таблица: Экспонаты (идентификатор, внешний ключ на справочник, название и числовой показатель – ценность экспоната)

CSV-файлы с данными таблицами указаны на рисунках 1 и 2:

```
mus_id;mus_name;  
1;"Музей ВОВ";  
2;"Музей военной техники";  
3;"Музей Первой Мировой войны";  
4;"Музей военной формы";
```

Рисунок 1 – Содержимое файла museum.csv

```
expo_id;mus_id;expo_name;expo_value  
1;1;"Знамя Победы";5000000  
2;1;"Часы маршала";850000  
3;1;"Планшет Багратион";1200000  
4;2;"Танк Т-34-85";75000000  
5;2;"Катюша БМ-13";46000000  
6;2;"Истребитель Як-3";98000000  
7;3;"Винтовка Мосина";45000  
8;3;"Георгиевский крест";280000  
9;3;"Каска Адриана";35000  
10;4;"Мундир Маршала";650000  
11;4;"Шинель 1943";28000  
12;4;"Гимнастерка РИА";95000
```

Рисунок 2 – Содержимое файла expo.csv

2. Реализация алгоритма

2.1. Создание классов-моделей

Создадим классы-модели для обеих таблиц, в каждом из которых:

- Поля реализованы через свойства, и у числового поля есть валидация в set-аксессоре
- Два конструктора: с параметрами и по умолчанию
- Переопределение метода ToString()
- XML-комментарии для класса и публичных членов

Реализации данных классов представлены на рисунках 3-5:

```

class Museum
{
    Ссылка: 2
    public int Id { get; set; }
    Ссылка: 2
    public string Name { get; set; }
    /// <summary>
    /// Конструктор с параметрами
    /// </summary>
    /// <param name="id"></param>
    /// <param name="name"></param>
    Ссылка: 1
    public Museum(int id, string name)
    {
        Id = id;
        Name = name;
    }
    /// <summary>
    /// Конструктор по умолчанию
    /// </summary>
    public Museum() : this(0, "") { }

    Ссылка: 0
    public override string ToString() => $"{Id} : {Name}";
}

```

Рисунок 3 – Реализация класса Museum

```

class Expo
{
    Ссылка: 2
    public int Id { get; set; }
    Ссылка: 1
    public int MusId { get; set; }
    Ссылка: 2
    public string Name { get; set; }
    public double _value;
    /// <summary>
    /// Ценность экспоната (она не может быть отрицательной)
    /// </summary>
    Ссылка: 2
    public double Value
    {
        get => _value;
        set
        {
            if (value < 0)
                throw new ArgumentException("Ценность объекта не может быть отрицательной");
            _value = value;
        }
    }
    /// <summary>
    /// Конструктор с параметрами
    /// </summary>
    /// <param name="id"></param>
    /// <param name="MusId"></param>
    /// <param name="name"></param>
    /// <param name="value"></param>
    Ссылка: 1
    public Expo(int id, int MusId, string name, double value)
    {
        Id = id;
        MusId = MusId;
        Name = name;
        Value = value;
    }
    /// <summary>
    /// Конструктор по умолчанию
    /// </summary>
    Ссылка: 0
    public Expo() : this(0, 0, "", 0) { }

    Ссылка: 0
    public override string ToString() => $"{Id} : Экспонат '{Name}' музея {MusId} ценностью {Value} руб.";
}

```

Рисунок 4 – Реализация класса Expo

Составим схемы связи двух классов-моделей в нотации Чена и в форме Entity Relationship Diagram, где классы-модели – метавершины(рисунки 5 и 6)

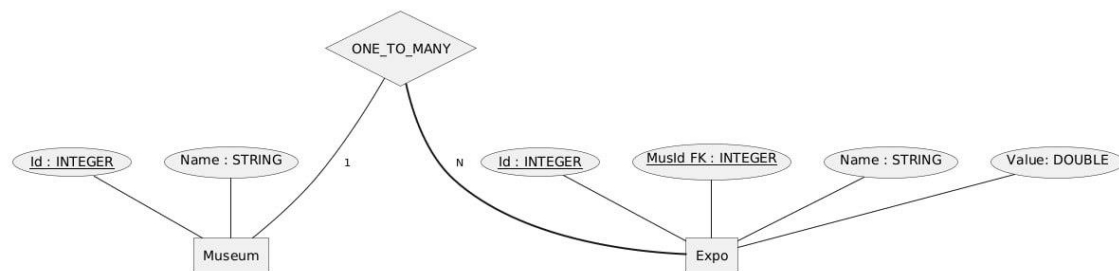


Рисунок 5 – Схема в форме нотации Чена

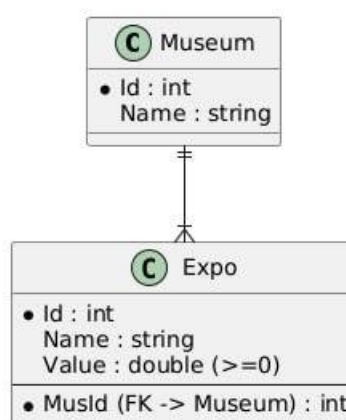


Рисунок 6 – Схема в форме Entity Relationship Diagram

2.2. Создание класса DataBaseManager

Создадим класс DataBaseManager, который будет инкапсулировать всё взаимодействие с SQLite – реализация данного класса со всеми функциями представлена в приложении А

2.3. Создание класса ReportBuilder

Реализация класса ReportBuilder, реализующего паттерн Fluent Interface для формирования и вывода отчётов(рисунки 7-8)

```
public class ReportBuilder
{
    private DatabaseManager _db;
    private string _sql = "";
    private string _title = "";
    private string[] _headers = Array.Empty<string>();
    private int[] _widths = Array.Empty<int>();
    private bool _numbered = false;
    private string _footer = "";

    Ссылка 3
    public ReportBuilder(DatabaseManager db) { _db = db; }

    /// <summary>
    /// Устанавливает SQL-запрос для получения данных отчёта
    /// </summary>
    /// <param name="sql"></param>
    Ссылка 3
    public ReportBuilder Query(string sql) { _sql = sql; return this; }

    /// <summary>
    /// Заголовок отчёта
    /// </summary>
    /// <param name="title"></param>
    Ссылка 3
    public ReportBuilder Title(string title) { _title = title; return this; }

    /// <summary>
    /// Устанавливает названия колонок таблицы
    /// </summary>
    /// <param name="columns"></param>
    Ссылка 3
    public ReportBuilder Header(params string[] columns) { _headers = columns; return this; }

    /// <summary>
    /// Устанавливает ширину каждой колонки
    /// </summary>
    /// <param name="widths"></param>
    Ссылка 3
    public ReportBuilder ColumnWidths(params int[] widths) { _widths = widths; return this; }

    /// <summary>
    /// Добавление нумерации строк
    /// </summary>
    Ссылка 1
    public ReportBuilder Numbered() { _numbered = true; return this; }

    /// <summary>
    /// Добавление сообщения в конце
    /// </summary>
    /// <param name="footer"></param>
    /// <returns></returns>
    Ссылка 1
    public ReportBuilder Footer(string footer) { _footer = footer; return this; }
```

Рисунок 7 – Атрибуты и промежуточные методы ReportBuilder (включая задание по варианту)

```
public string Build()
{
    var (columns, rows) = _db.ExecuteQuery(_sql);
    var sb = new StringBuilder();

    // Заголовок
    if (!string.IsNullOrEmpty(_title))
    {
        sb.AppendLine();
        sb.AppendLine($"=== {_title} ===");
    }

    string[] displayHeaders = _headers.Length > 0 ? _headers : columns;
    int colCount = displayHeaders.Length;
    int numWidth = _numbered ? 5 : 0;

    // Ветка таблицы
    if (_numbered) sb.Append($"{numWidth}");
    for (int i = 0; i < colCount; i++)
        sb.Append(displayHeaders[i].PadRight(_widths.Length > i ? _widths[i] : 45));

    sb.AppendLine();

    // Разделительная линия
    int totalWidth = numWidth;
    for (int i = 0; i < colCount; i++)
        totalWidth += (_widths.Length > i ? _widths[i] : 20);
    sb.AppendLine(new string('-', totalWidth));

    // Данные строк
    for (int r = 0; r < rows.Count; r++)
    {
        if (_numbered) sb.Append((r + 1).ToString().PadRight(numWidth));
        for (int c = 0; c < rows[r].Length && c < colCount; c++)
        {
            sb.Append(rows[r][c].PadRight(_widths.Length > c ? _widths[c] : 45));
        }
        sb.AppendLine();
    }

    // Второе сообщение
    if (!string.IsNullOrEmpty(_footer))
    {
        sb.AppendLine(new string('-', totalWidth));
        sb.AppendLine($"*{_footer} {rows.Count}*");
    }

    return sb.ToString();
}
```

Рисунок 8 – Терминальный метод Build() класса ReportBuilder

Создадим функции для реализации 3 обязательных отчётов в Program.cs (рисунок 9)

```
static void ReportAllExposWithMuseums(DatabaseManager db)
{
    string sql = @"
    SELECT e.Expo_name, m.museum_name, e.value_k
    FROM Expos e
    JOIN museums m ON e.museum_id = m.museum_id
    ORDER BY e.Expo_name ASC";

    new ReportBuilder(db)
        .Query(sql)
        .Title("ПОЛНЫЙ СПИСОК ЭКСПОНАТОВ")
        .Header("Экспонат", "Музей", "Стоимость (т.р.)")
        .ColumnWidths(30, 25, 30)
        .Footer("Всего экспонатов:")
        .Numbered()
        .Print();
}

// Ссылка 1
static void ReportExposCountByMuseum(DatabaseManager db)
{
    string sql = @"
    SELECT m.museum_name, COUNT(*) AS cnt
    FROM Expos e
    JOIN museums m ON e.museum_id = m.museum_id
    GROUP BY m.museum_name
    ORDER BY cnt DESC";

    new ReportBuilder(db)
        .Query(sql)
        .Title("КОЛИЧЕСТВО ЭКСПОНАТОВ ПО МУЗЕЯМ")
        .Header("Название музея", "Кол-во объектов")
        .ColumnWidths(35, 35)
        .Print();
}

// Ссылка 1
static void ReportAverageValueByMuseum(DatabaseManager db)
{
    string sql = @"
    SELECT m.museum_name, ROUND(AVG(e.value_k), 2) as avg_val
    FROM Expos e
    JOIN museums m ON e.museum_id = m.museum_id
    GROUP BY m.museum_name
    ORDER BY avg_val DESC";

    new ReportBuilder(db)
        .Query(sql)
        .Title("СРЕДНЯЯ ЦЕННОСТЬ ФОНДОВ")
        .Header("Музей", "Средняя цена (т.р.)")
        .ColumnWidths(35, 20)
        .Print();
}
```

Рисунок 9 – Функции создания отчётов классом ReportBuilder

Пример вывода первого отчёта представлен на рисунке 10:

```
=== МЕНЮ ОТЧЕТОВ ===
1. Полный список экспонатов с названиями музеев
2. Количество экспонатов в каждом музее
3. Средняя ценность экспонатов по музеям
0. Назад

Выберите отчет: 1

=== ПОЛНЫЙ СПИСОК ЭКСПОНАТОВ ===
```

№	Экспонат	Музей	Стоимость (т.р.)
1	"Винтовка Мосина"	"Музей Первой Мировой войны"	45
2	"Георгиевский крест"	"Музей Первой Мировой войны"	280
3	"Гимнастерка РИА"	"Музей военной формы"	95
4	"Знамя Победы"	"Музей ВОВ"	5000
5	"Истребитель Як-3"	"Музей военной техники"	98000
6	"Каска Адриана"	"Музей Первой Мировой войны"	35
7	"Катюша БМ-13"	"Музей военной техники"	46000
8	"Мундир Маршала"	"Музей военной формы"	650
9	"Планшет Багратион"	"Музей ВОВ"	1200
10	"Танк Т-34-85"	"Музей военной техники"	75000
11	"Часы маршала"	"Музей ВОВ"	850
12	"Шинель 1943"	"Музей военной формы"	28

Рисунок 10 – Вывод в консоль первого отчёта

Составим Activity Diagram для процесса создания первого отчёта:

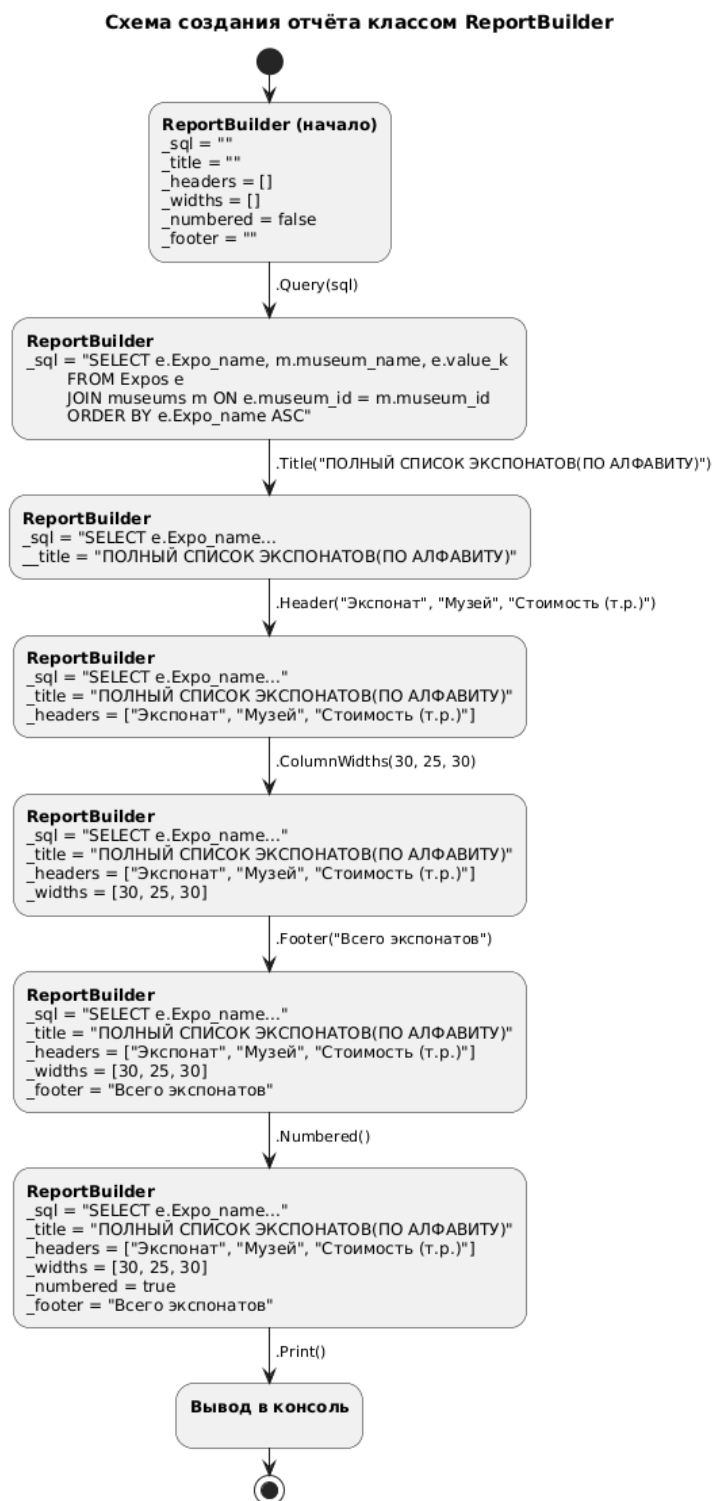


Рисунок 11 – Activity Diagram работы класса ReportBuilder для создания первого отчёта

2.4. Создание консольного меню

Создадим интерактивное меню в Program.cs, реализация которого представлена на рисунках 12-14:

```
static void Main(string[] args)
{
    var db = new DatabaseManager(dbPath);
    db.InitializeDatabase(museumsCsv, exposCsv);

    bool exit = false;
    while (!exit)
    {
        Console.Clear();
        Console.WriteLine("=== УЧЕТ МУЗЕЙНЫХ ЭКСПОНАТОВ ===");
        Console.WriteLine("1. Список всех музеев");
        Console.WriteLine("2. Список всех экспонатов");
        Console.WriteLine("3. Добавить новый экспонат");
        Console.WriteLine("4. Обновить сущ. экспонат");
        Console.WriteLine("5. Удалить сущ. экспонат");
        Console.WriteLine("6. Сформировать отчеты");
        Console.WriteLine("7. Экспорт данных в CSV");
        Console.WriteLine("8. Выход");
        Console.WriteLine("\nВыберите пункт меню: ");

        switch (Console.ReadLine())
        {
            case "1": ListMuseums(db); break;
            case "2": ListExpos(db); break;
            case "3": CreateNewExpo(db); break;
            case "4": UpdateExExpo(db); break;
            case "5": DeleteExExpo(db); break;
            case "6": ShowReportsMenu(db); break;
            case "7": ExportToFiles(db); break;
            case "8": exit = true; break;
            default: Console.WriteLine("Неверный ввод."); break;
        }

        if (!exit)
        {
            Console.WriteLine("\nНажмите любую клавишу...");
            Console.ReadKey();
        }
    }
}
```

Рисунок 12 – Функция main с интерактивным меню

```
static void ListMuseums(DatabaseManager db)
{
    var list = db.GetAllMuseums();
    Console.WriteLine("\n--- Список музеев ---");
    foreach (var m in list) Console.WriteLine($"{m.Id} | Название: {m.Name}");
}

Ссылка 1
static void ListExpos(DatabaseManager db)
{
    var list = db.GetAllExpos();
    Console.WriteLine("\n--- Реестр экспонатов ---");
    Console.WriteLine($"{ID",-5} {Музей ID",-10} {Наименование",-25} {Оценка (т.р.)",-15}");
    foreach (var e in list)
        Console.WriteLine($"{e.Id,-5} {e.MusId,-10} {e.Name,-25} {e.Value,-15}");
}

Ссылка 1
static void CreateNewExpo(DatabaseManager db)
{
    Console.WriteLine("\n--- Регистрация нового экспоната ---");
    ListMuseums(db);

    Console.Write("Введите ID музея: ");
    if (!int.TryParse(Console.ReadLine(), out int musId))
    {
        Console.WriteLine("Ошибка: ID музея должен быть целым числом.");
        return;
    }

    Console.Write("Название экспоната: ");
    string name = Console.ReadLine() ?? "Без названия";

    Console.Write("Оценочная стоимость (тыс. руб.): ");
    if (!int.TryParse(Console.ReadLine(), out int val))
    {
        Console.WriteLine("Ошибка: стоимость должна быть целым числом.");
        return;
    }

    db.AddExpo(new Expo(0, musId, name, val));
    Console.WriteLine("Экспонат успешно добавлен.");
}
```

Рисунок 13 – Функции вывода музеев, вывода и создания экспонатов

```

static void UpdateExExpo(DatabaseManager db)
{
    Console.WriteLine("— Редактирование экспоната —");
    Console.Write("Введите ID экспоната: ");
    if (!int.TryParse(Console.ReadLine(), out int id))
    {
        Console.WriteLine("Ошибка: ID экспоната должен быть целым числом.");
        return;
    }

    var expo = db.GetExpoById(id);
    if (expo == null)
    {
        Console.WriteLine("Экспонат не найден.");
        return;
    }

    Console.WriteLine($"Текущие данные: {expo}");
    Console.WriteLine("(Нажмите Enter, чтобы не менять значение)");

    Console.Write($"Название [{expo.Name}]: ");
    string input = Console.ReadLine();
    if (!string.IsNullOrEmpty(input)) expo.Name = input;

    Console.Write($"ID музея [{expo.MusId}]: ");
    input = Console.ReadLine();
    if (!string.IsNullOrEmpty(input)) expo.MusId = int.Parse(input);

    Console.Write($"Стоимость [{expo.Value}]: ");
    input = Console.ReadLine();
    if (!string.IsNullOrEmpty(input)) expo.Value = double.Parse(input.Replace(',', '.'));

    db.UpdateExpo(expo);
    Console.WriteLine("Готово! Данные обновлены.");
}

Семейка 1
static void DeleteExExpo(DatabaseManager db)
{
    Console.WriteLine("— Удаление экспоната —");
    Console.Write("Введите ID экспоната: ");
    if (!int.TryParse(Console.ReadLine(), out int id))
    {
        Console.WriteLine("Ошибка: ID экспоната должен быть целым числом.");
        return;
    }

    var expo = db.GetExpoById(id);
    if (expo == null)
    {
        Console.WriteLine("Экспонат не найден.");
        return;
    }

    Console.Write($"Удалить «{expo.Name}»? (д/н): ");
    if (Console.ReadLine().ToLower() == "д")
    {
        db.DeleteExpo(id);
        Console.WriteLine("Удалено.");
    }
    else
        Console.WriteLine("Отмена.");
}

```

Рисунок 14 – Функции изменения и удаления экспонатов

3. Вывод программы

На рисунках 15-22 представлены примеры работы программы:

```
=== УЧЕТ МУЗЕЙНЫХ ЭКСПОНАТОВ ===
1. Список всех музеев
2. Список всех экспонатов
3. Добавить новый экспонат
4. Обновить сущ. экспонат
5. Удалить сущ. экспонат
6. Сформировать отчеты
7. Экспорт данных в CSV
0. Выход

Выберите пункт меню: 1

--- Список музеев ---
ID: 1 | Название: "Музей ВОВ"
ID: 2 | Название: "Музей военной техники"
ID: 3 | Название: "Музей Первой Мировой войны"
ID: 4 | Название: "Музей военной формы"
```

Рисунок 15 – Пример вывода всех музеев

```
Выберите пункт меню: 2

--- Реестр экспонатов ---
ID    Музей ID    Наименование                Оценка (т.р.)
1      1      "Знамя Победы"             5000
2      1      "Часы маршала"              850
3      1      "Планшет Багратион"        1200
4      2      "Танк Т-34-85"             75000
5      2      "Катюша БМ-13"            46000
6      2      "Истребитель Як-3"         98000
7      3      "Винтовка Мосина"          45
8      3      "Георгиевский крест"       280
9      3      "Каска Адриана"            35
10     4      "Мундир Маршала"           650
11     4      "Шинель 1943"              28
12     4      "Гимнастерка РИА"          95
```

Рисунок 16 – Вывод всех экспонатов

```
Выберите пункт меню: 3

--- Регистрация нового экспоната ---

--- Список музеев ---
ID: 1 | Название: "Музей ВОВ"
ID: 2 | Название: "Музей военной техники"
ID: 3 | Название: "Музей Первой Мировой войны"
ID: 4 | Название: "Музей военной формы"
Введите ID музея: 3
Название экспоната: Немецкий противогаз
Оценочная стоимость (тыс. руб.): 15
Экспонат успешно добавлен.

Нажмите любую клавишу...
```

Рисунок 17 – Добавление нового экспоната

```

Выберите пункт меню: 4
--- Редактирование экспоната ---
Введите ID экспоната: 13
Текущие данные: 13 : Экспонат 'Немецкий противогаз' музея 3 ценностью 15 тыс. руб.
(Нажмите Enter, чтобы не менять значение)
Название [Немецкий противогаз]:
ID музея [3]:
Стоимость [15]: 25
Готово! Данные обновлены.

```

Рисунок 18 – Изменение данных экспоната

```

Выберите пункт меню: 5
--- Удаление экспоната ---
Введите ID экспоната: 13
Удалить <Немецкий противогаз>? (д/н): д
Удалено.

```

Рисунок 19 – Удаление экспоната

Пример создания первого отчёта был представлен в разделе создания класса ReportBuilder.

```

Выберите отчет: 2

=== КОЛИЧЕСТВО ЭКСПОНАТОВ ПО МУЗЕЯМ ===
Название музея                Кол-во объектов
-----
"Музей военной формы"         3
"Музей военной техники"        3
"Музей Первой Мировой войны"   3
"Музей ВОВ"                    3

```

Рисунок 20 – Создание второго отчёта

```

=== МЕНЮ ОТЧЕТОВ ===
1. Полный список экспонатов с названиями музеев
2. Количество экспонатов в каждом музее
3. Средняя ценность экспонатов по музеям
0. Назад

Выберите отчет: 3

=== СРЕДНЯЯ ЦЕННОСТЬ ФОНДОВ ===
Музей                          Средняя цена (т.р.)
-----
"Музей военной техники"        73000
"Музей ВОВ"                    2350
"Музей военной формы"          257,67
"Музей Первой Мировой войны"   120

```

Рисунок 21 – Создание третьего отчёта

Вывод: в результате выполнения работы были получены навыки создания на C# консольного приложения, способного взаимодействовать с CSV-файлами и базами данных.

СПИСОК ИСПОЛЬЗУЕМЫХ ИСТОЧНИКОВ

1. Быков, А. Ю. Решение задач на языках программирования Си и Си++ : методические указания / А. Ю. Быков. — Москва : МГТУ им. Н.Э. Баумана, 2017. — 248 с. — ISBN 978-5-7038-4577-6. — Текст : электронный // Лань : электронно-библиотечная система. — URL: <https://e.lanbook.com/book/103505>
2. Каширин, И. Ю. От Си к Си++ : учебное пособие / И. Ю. Каширин, В. С. Новичков. — 2-е изд., стер. — Москва : Горячая линия-Телеком, 2012. — 334 с. — ISBN 978-5-9912-0259-6. — Текст : электронный // Лань : электронно-библиотечная система. — URL: <https://e.lanbook.com/book/5161>
3. Быков А. Ю. Решение задач на языках программирования Си и Си++ : метод. указания к выполнению лаб. работ / Быков А. Ю. ; МГТУ им. Н. Э. Баумана. - М. : Изд-во МГТУ им. Н. Э. Баумана, 2017. - 244 с. : ил. - ISBN 978-5-7038-4577-6.
4. Иванова Г. С., Ничушкина Т. Н. Объектно-ориентированное программирование : учебник для вузов / Иванова Г. С., Ничушкина Т. Н. ; общ. ред. Иванова Г. С. - М. : Изд-во МГТУ Н.Э.Баумана, 2002. <http://progbook.ru/technologiya-programmirovaniya/582-ivanova-tehnologiya-programmirovaniya.html>
5. Иванова Г. С., Ничушкина Т. Н. Объектно-ориентированное программирование : учебник для вузов / Иванова Г. С., Ничушкина Т. Н. ; общ. ред. Иванова Г. С. - М. : Изд-во МГТУ им. Н. Э. Баумана, 2014. - 455 с. : ил. - Библиогр.: с. 450. - ISBN 978-5-7038-3921-8.
6. Подбельский В.В. Язык Си++: Учебное пособие. – М.: Финансы и статистика, 2003. <http://progbook.ru/c/737-podbelskii-programmiovaniye-na-yazyke-si.html>.
7. Акулов О.А., Медведев Н.В. Информатика. Базовый курс. – М.: Омега-Л, 2006. <http://razym.ru/naukaobraz/obrazov/151874-akulov-oa-medvedev-nv-informatika-bazovyy-kurs.html>

8. Вычислительные методы и программирование. МГУ им. М.В. Ломоносова. ISSN 1726-3522. Журнал входит в 1-й уровень Белого списка научных журналов Минобрнауки России. <https://num-meth.ru/index.php/journal/index>

Дополнительные материалы

1. Иванова Г.С. Технология программирования: Учебник для вузов. – М.: Изд. МГТУ им. Ахо А.В., Хопкрофт Д.Э., Ульман Д.Д. Структуры данных и алгоритмы. – М., Вильямс, 2003. <http://razym.ru/naukaobraz/obrazov/181547-aho-a-ulman-d-hopkroft-d-struktury-dannyh-i-algoritmy.html>
2. Дейтел Х.М., Дейтел П.Дж. Как программировать на C++. – М.: Бином, 2001.
3. Т. Кормен Т., Лейзерсон Ч., Ривест Р. Алгоритмы: построение и анализ. – М. МЦНМО, 2005. <http://rutracker.org/forum/viewtopic.php?t=533181>
4. Джосьютис Н. C++ Стандартная библиотека для профессионалов. – СПб.: Питер, 2004. http://progbook.ru/c/178-dzhosyutis_c_standartnaya_biblioteka.html
5. Подбельский В.В. Стандартный Си++: Учебное пособие. – М.: Финансы и статистика, 2008.
6. Объектно-ориентированное программирование в C++: пер. с англ. / Лафоре Р. - 4-е изд. - СПб.: Питер, 2004. - 923 с. - (Классика computer science). - ISBN 5-94723-302-9.
7. Т. Кормен Т., Лейзерсон Ч., Ривест Р. Алгоритмы: построение и анализ. – М. МЦНМО, 2005.
8. Г. Шилдт. C++. Базовый курс, 3-е издание: Пер с англ. – М.: Издательский дом «Вильямс», 2011. – 624 с.
9. Павловская Т. А. C/C++. Программирование на языке высокого уровня: учебник для вузов / Павловская Т. А. - СПб.: Питер, 2003. - 460 с. - (Учебник для вузов). - ISBN 5-94723-568-4.
10. Бесплатные образовательные программы партнера (VK): <https://education.vk.company/students>

ПРИЛОЖЕНИЕ А

Листинг класса DataBaseManager

```
public class DataBaseManager
{
    private string _connectionString;

    public DataBaseManager(string dbPath)
    {
        _connectionString = $"Data Source={dbPath}";
    }

    /// <summary>
    /// Создает таблицы и выполняет первичный импорт данных из CSV-файлов
    /// </summary>
    public void InitializeDatabase(string museumsCsvPath, string ExposCsvPath)
    {
        using var conn = new SqlConnection(_connectionString);
        conn.Open();
        var cmd = conn.CreateCommand();
        cmd.CommandText = @"
CREATE TABLE IF NOT EXISTS museums (
    museum_id INTEGER PRIMARY KEY AUTOINCREMENT,
    museum_name TEXT NOT NULL
);
CREATE TABLE IF NOT EXISTS Expos (
    Expo_id INTEGER PRIMARY KEY AUTOINCREMENT,
    museum_id INTEGER NOT NULL,
    Expo_name TEXT NOT NULL,
    value_k INTEGER NOT NULL,
    FOREIGN KEY (museum_id) REFERENCES museums(museum_id)
);";
        cmd.ExecuteNonQuery();

        if (GetAllMuseums().Count == 0 && File.Exists(museumsCsvPath))
        {
            ImportMuseumsFromCsv(museumsCsvPath);
            Console.WriteLine($"Загружены музеи из {Path.GetFileName(museumsCsvPath)}");
        }

        if (GetAllExpos().Count == 0 && File.Exists(ExposCsvPath))
        {
            ImportExposFromCsv(ExposCsvPath);
            Console.WriteLine($"Загружены экспонаты из {Path.GetFileName(ExposCsvPath)}");
        }
    }

    /// <summary>
    /// Загружает записи о музеях из CSV-файла
    /// </summary>
    private void ImportMuseumsFromCsv(string path)
    {
        using var conn = new SqlConnection(_connectionString);
        conn.Open();
        string[] lines = File.ReadAllLines(path);
        for (int i = 1; i < lines.Length; i++)
        {
            string[] parts = lines[i].Split(';');
            if (parts.Length < 2) continue;

            var cmd = conn.CreateCommand();
            cmd.CommandText = "INSERT INTO museums (museum_id, museum_name) VALUES (@id, @name)";
            cmd.Parameters.AddWithValue("@id", int.Parse(parts[0]));
            cmd.Parameters.AddWithValue("@name", parts[1]);
            cmd.ExecuteNonQuery();
        }
    }
}
```

```

    }
}

/// <summary>
/// Загружает записи об экспонатах из CSV-файла
/// </summary>
private void ImportExposFromCsv(string path)
{
    using var conn = new SqliteConnection(_connectionString);
    conn.Open();
    string[] lines = File.ReadAllLines(path);
    for (int i = 1; i < lines.Length; i++)
    {
        string[] parts = lines[i].Split(';');
        if (parts.Length < 4) continue;

        var cmd = conn.CreateCommand();
        cmd.CommandText = @"INSERT INTO Expos (Expo_id, museum_id, Expo_name, value_k)
                           VALUES (@eId, @mId, @name, @val)";

        cmd.Parameters.AddWithValue("@eId", int.Parse(parts[0]));
        cmd.Parameters.AddWithValue("@mId", int.Parse(parts[1]));
        cmd.Parameters.AddWithValue("@name", parts[2]);
        cmd.Parameters.AddWithValue("@val", int.Parse(parts[3]));

        cmd.ExecuteNonQuery();
    }
}

/// <summary>
/// Возвращает полный список музеев
/// </summary>
public List<Museum> GetAllMuseums()
{
    var result = new List<Museum>();
    using var conn = new SqliteConnection(_connectionString);
    conn.Open();
    var cmd = conn.CreateCommand();
    cmd.CommandText = "SELECT museum_id, museum_name FROM museums ORDER BY museum_id";
    using var reader = cmd.ExecuteReader();
    while (reader.Read())
        result.Add(new Museum(reader.GetInt32(0), reader.GetString(1)));
    return result;
}

/// <summary>
/// Возвращает полный список экспонатов
/// </summary>
public List<Expo> GetAllExpos()
{
    var result = new List<Expo>();
    using var conn = new SqliteConnection(_connectionString);
    conn.Open();
    var cmd = conn.CreateCommand();
    cmd.CommandText = "SELECT Expo_id, museum_id, Expo_name, value_k FROM Expos ORDER
BY Expo_id";
    using var reader = cmd.ExecuteReader();
    while (reader.Read())
        result.Add(new Expo(reader.GetInt32(0), reader.GetInt32(1),
reader.GetString(2), reader.GetInt32(3)));
    return result;
}

/// <summary>
/// Добавляет новую запись об экспонате
/// </summary>
public void AddExpo(Expo Expo)
{
    using var conn = new SqliteConnection(_connectionString);
    conn.Open();
    var cmd = conn.CreateCommand();
    cmd.CommandText = "INSERT INTO Expos (museum_id, Expo_name, value_k) VALUES (@mId,
@name, @val)";

```

```

        cmd.Parameters.AddWithValue("@mId", Expo.MusId);
        cmd.Parameters.AddWithValue("@name", Expo.Name);
        cmd.Parameters.AddWithValue("@val", Expo.Value);
        cmd.ExecuteNonQuery();
    }

    /// <summary>
    /// Обновляет существующую запись об экспонате
    /// </summary>
    public void UpdateExpo(Expo Expo)
    {
        using var conn = new SqliteConnection(_connectionString);
        conn.Open();
        var cmd = conn.CreateCommand();
        cmd.CommandText = "UPDATE Expos SET museum_id=@mId, Expo_name=@name, value_k=@val
WHERE Expo_id=@id";
        cmd.Parameters.AddWithValue("@id", Expo.Id);
        cmd.Parameters.AddWithValue("@mId", Expo.MusId);
        cmd.Parameters.AddWithValue("@name", Expo.Name);
        cmd.Parameters.AddWithValue("@val", Expo.Value);
        cmd.ExecuteNonQuery();
    }

    /// <summary>
    /// Удаляет экспонат по идентификатору
    /// </summary>
    public void DeleteExpo(int id)
    {
        using var conn = new SqliteConnection(_connectionString);
        conn.Open();
        var cmd = conn.CreateCommand();
        cmd.CommandText = "DELETE FROM Expos WHERE Expo_id=@id";
        cmd.Parameters.AddWithValue("@id", id);
        cmd.ExecuteNonQuery();
    }

    /// <summary>
    /// Ищет экспонат по идентификатору
    /// </summary>
    public Expo GetExpoById(int id)
    {
        using var conn = new SqliteConnection(_connectionString);
        conn.Open();
        var cmd = conn.CreateCommand();
        cmd.CommandText = "SELECT Expo_id, museum_id, Expo_name, value_k FROM Expos WHERE
Expo_id=@id";
        cmd.Parameters.AddWithValue("@id", id);
        using var reader = cmd.ExecuteReader();
        if (reader.Read())
            return new Expo(reader.GetInt32(0), reader.GetInt32(1), reader.GetString(2),
reader.GetInt32(3));
        return null;
    }

    /// <summary>
    /// Выполняет SQL-запрос для отчетов
    /// </summary>
    public (string[] columns, List<string[]> rows) ExecuteQuery(string sql)
    {
        using var conn = new SqliteConnection(_connectionString);
        conn.Open();
        var cmd = conn.CreateCommand();
        cmd.CommandText = sql;
        using var reader = cmd.ExecuteReader();

        string[] columns = new string[reader.FieldCount];
        for (int i = 0; i < reader.FieldCount; i++)
            columns[i] = reader.GetName(i);

        var rows = new List<string[]>();
        while (reader.Read())
        {
            string[] row = new string[reader.FieldCount];

```

```

        for (int i = 0; i < reader.FieldCount; i++)
            row[i] = reader.GetValue(i)?.ToString() ?? "";
        rows.Add(row);
    }
    return (columns, rows);
}

public void ExportToCsv(string museumPath, string expoPath)
{
    museumPath = Path.Combine(AppContext.BaseDirectory, museumPath);
    expoPath = Path.Combine(AppContext.BaseDirectory, expoPath);
    var mLines = new List<string> { "museum_id; museum_name" };
    foreach (var m in GetAllMuseums())
        mLines.Add($"{m.Id}; {m.Name}");
    File.WriteAllLines(museumPath, mLines.ToArray());

    var eLines = new List<string> { "Expo_id; museum_id; Expo_name; value" };
    foreach (var e in GetAllExpos())
        eLines.Add($"{e.Id}; {e.MusId}; {e.Name}; {e.Value}");
    File.WriteAllLines(expoPath, eLines.ToArray());
}
}

```